

FUNCTION -->

- Inbuild function -- print(), type(), id(), sqrt(), max(), min(), sqrt(), ceil(), etc
- User defined function
- function is collection of statement
- 2 main property of the function is -- define the function & calling the function
- function always define with def & function always declares as () difference between variable & function || a - variable || b() - function

```
def greet():  
    print('hello')  
    print('good morning')  
    # when we run the code we havent got any output
```

```
def greet():  
    print('hello')  
    print('good morning')  
greet() #if you need call multiple times
```

```
hello  
good morning
```

```
def greet():  
    print('hello')  
    print('good morning')  
greet() #if you need call multiple times
```

```
def greet():  
    print('hello')  
    print('good morning')  
greet() #if you need call multiple times
```

```
hello  
good morning  
hello  
good morning
```

```
def greet():  
    print('hello')  
    print('good noon')  
greet()
```

```
greet()
```

```
hello  
good noon  
hello  
good noon
```

```

def greet():
    print('hello')
    print('good noon')
greet()

print()

greet()

hello
good noon

hello
good noon

def add(x,y):
    c=x+y
    print(c)
add(5,6)

11

def add(x,y,z):
    c=x+y+z
    print(c)
add(1,4,5)

10

def greet():
    print('hello')
    print('good noon')
greet()

def add(x,y):
    c=x+y
    print(c)
add(5,4)
# you can create mutliple function and call them as many time as you want

hello
good noon
9

def greet():
    print('hello')
    print('good noon')

def add(x,y):
    c=x+y

```

```

    print(c)

def sub(x,y,z):
    d = x-y-z
    print(d)

greet()
add(5,4)
sub(10,2,4)
# you can create mutliple function and call them as many time as you want

hello
good noon
9
4

def greet():
    print('hello')
    print('good noon')
greet()

def add(x,y):
    c=x+y
    return c

result = add(5,4)
print(result)

hello
good noon
9

def greet():
    print('hello')
    print('good noon')

def add(x,y,z):
    c=x+y+z
    return c

greet()

result = add(5,4,3)
print(result)

hello
good noon
12

```

- as a function we have 2 choice 1- whenever we call the function . function is do the task for you greet() & add() 2- we have another type of function it will return you the value

```
def add_sub(x,y): # what if i want to return 2 values add_sub & i want
to return 2 values & function can accept multiple value
    c= x+y
    d= x-y
    return c, d

add_sub(4,5)
(9, -1)

def add_sub(x,y): # what if i want to return 2 values add_sub & i want
to return 2 values & function can accept multiple value
    c= x+y
    d= x-y
    return c, d

result = add_sub(4,5)

print(result)
print(type(result))
(9, -1)
<class 'tuple'>

def add_sub(x,y):
    c= x+y
    d= x-y
    return c, d

result1,result2 = add_sub(5,4)

print(result1,result2)

print(type(result1))
print(type(result2))
9 1
<class 'int'>
<class 'int'>

a,b = 6,7
```

- function are always reuseable
- in one code you can write multiple function as well
- FUNCTION ARGUMENT
- How to pass parameter to a function & what happend to the variable when you pass to a function & if you modify the value then what happen
- every code check with debug

```
def update():  
    x = 8  
    print(x)  
update()
```

8

```
def update(): #update function take the value from the user  
    x = 8  
    print(x)  
update(8)
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
Cell In[17], line 4  
      2     x = 8  
      3     print(x)  
----> 4 update(8)
```

TypeError: update() takes 0 positional arguments but 1 was given

```
def update(x): #update function take the value from the user  
    x = 8  
    print(x)
```

update(10)

8

```
def update(x): # user want to update the value from 8 to 10  
    x = 8  
    print(x)
```

update(10)

8

```
def update(x):  
    x = 8  
    print(x)
```

```
a = 10  
update(a)  
print(a)
```

8

10

```
def update(x):  
    x = 8
```

```

    #print(x)

a = 10
update(a)
print(a) # this print will update 8 to 5
10

def change(a):
    a = a+ 10
    print('inside the fun a =',a)

x = 10
print('x before calling:', x)
change(x)
print('x after calling:', x)

x before calling: 10
inside the fun a = 20
x after calling: 10

def change(a):
    a = a+ 10
    print('inside the fun a =',a)

a = 10
print('a before calling:', a)
change(a)
print('a after calling:', a)

a before calling: 10
inside the fun a = 20
a after calling: 10

def change(a):
    print('This is original a',id(a))
    a = a+ 10
    print('This is the new a =',a)
    print('inside the fun a =',a)

a = 10
print('a before calling:', a)
print('This is main a:',id(a))
change(a)
print('a after calling:', a)

a before calling: 10
This is main a: 140707061867720
This is original a 140707061867720
This is the new a = 20

```

inside the fun a = 20
a after calling: 10

```
def change(a):  
    print('This is original a',id(a))  
    a = a+ 10  
    print('This is the new a =',id(a))  
    print('inside the fun a =',a)
```

```
a = 10  
print('a before calling:', a)  
print('This is main a:',id(a))  
change(a)  
print('a after calling:', a)
```

a before calling: 10
This is main a: 140707061867720
This is original a 140707061867720
This is the new a = 140707061868040
inside the fun a = 20
a after calling: 10

```
def change(a):  
    #print('This is original a',id(a))  
    a = a+ 10  
    print('This is the new a =',id(a))  
    print('inside the fun a =',a)
```

```
a = 10  
print('a before calling:', a)  
print('This is main a:',id(a))  
change(a)  
print('a after calling:', a)  
print('This is original a',id(a))
```

a before calling: 10
This is main a: 140707061867720
This is the new a = 140707061868040
inside the fun a = 20
a after calling: 10
This is original a 140707061867720

```
def change(lst):  
    lst[0] = lst[0]+10  
    print('inside fun =', lst)
```

```
lst = [10]  
print('Before calling:', lst)  
change(lst)  
print('After calling:',lst)
```

Before calling: [10]
inside fun = [20]
After calling: [20]

```
def update(x):  
    x = 8  
    print('x : ', x)
```

```
a = 10  
update(a)  
print('a : ',a)
```

```
x : 8  
a : 10
```

```
def update(x):  
    print(id(x))  
    x = 8  
    #print(id(x))  
    print('x', x)
```

```
a = 10  
print(id(a))  
update(a)  
print('a',a)
```

```
140707061867720  
140707061867720  
x 8  
a 10
```

```
def update(x):  
    #print(id(x))  
    x = 8  
    print(id(x))  
    print('x', x)
```

```
a = 10  
print(id(a))  
update(a)  
print('a',a)
```

```
140707061867720  
140707061867656  
x 8  
a 10
```

```
def update(x):  
    x = 8  
  
    print(id(x))
```



```

    print('x', x)

a = 10
print(id(a))

update(a)
print('a',a)

# we will understand more when we learn more

140707061867720
140707061867656
x 8
a 10

def update(x):
    print(id(x))
    x = 8
    print(id(x))
    print('x', x)

a = 10
print(id(a))
update(a)
print('a',a)

140707061867720
140707061867720
140707061867656
x 8
a 10

def update(lst):
    print(id(lst))

    lst[1] = 25
    print(id(lst))
    print('x', lst)

lst = [10,20,30] #lets pass list hear
print(id(lst))
update(lst)
print('lst',lst)

2304087298624
2304087298624
2304087298624
x [10, 25, 30]
lst [10, 25, 30]

```

NO concept for pass by value in python (please refer the code below)

```
def modify_integer(x):
    x = 10
    print("Inside function:", x)

my_integer = 5
modify_integer(my_integer)
print("Outside function:", my_integer)

Inside function: 10
Outside function: 5

def modify_integer(x):
    x = 10
    print("Inside function:", x)
    print('Inside function:', id(x))

my_integer = 5
modify_integer(my_integer)
print("Outside function:", my_integer)
print('Outside function:', id(x))

Inside function: 10
Inside function: 140707061867720
Outside function: 5
Outside function: 140707061867720

def modify_integer(x):
    print('original Inside function:', id(x))
    x = 10
    print("Inside function:", x)
    print('Inside function:', id(x))

my_integer = 5
modify_integer(my_integer)
print("Outside function:", my_integer)
print('Outside function:', id(x))

original Inside function: 140707061867560
Inside function: 10
Inside function: 140707061867720
Outside function: 5
Outside function: 140707061867720
```

NO concept for pass by reference in python (please refer the code below)

```
def modify_list(my_list):
    my_list.append(4)
    print("Inside function:", my_list)

my_list = [1, 2, 3]
```

```

modify_list(my_list)
print("Outside function:", my_list)

Inside function: [1, 2, 3, 4]
Outside function: [1, 2, 3, 4]

def modify_list(my_list):
    print("original Inside function:", id(my_list))
    my_list.append(4)
    print("Inside function:", my_list)
    print("Inside function:", id(my_list))

my_list = [1, 2, 3]
modify_list(my_list)
print("Outside function:", my_list)
print("Outside function:", id(my_list))

original Inside function: 2304093089280
Inside function: [1, 2, 3, 4]
Inside function: 2304093089280
Outside function: [1, 2, 3, 4]
Outside function: 2304093089280

```

TYPE OF ARGUMENTS --> formal argument & actual argument

```

def add(a,b): # a & b called formal argument
    c = a+b
    print(c)

add(5,6) #5 and 6 we called as actual argument

11

def add(a,b,d,e): # a & b called formal argument
    c = a+b+d
    print(c)

add(5,6,7,8) #5 and 6 we called as actual argument

18

```

positional argument

```

def person(name,age):
    print(name)
    print(age)

person('nit', 22)

nit
22

```



```
last)
Cell In[45], line 5
      2     print(name)
      3     print(age-5)
----> 5 person(20,'nit')
```

```
Cell In[45], line 3, in person(name, age)
      1 def person(name,age):
      2     print(name)
----> 3     print(age-5)
```

TypeError: unsupported operand type(s) for -: 'str' and 'int'

```
def person(name,age):
    print(name)
    print(age-5)
```

```
person(age = 20, name = 'nit')
```

#this is called keyword arguments

```
nit
15
```

```
def person(name, age):
    print(name)
    print(age)
```

```
person(age = 20, name = 'nit')
```

#this is called keyword arguments

```
nit
20
```

```
def person(name, age, age2):
    print(name)
    print(age)
    print(age2)
```

```
person(age = 20, name = 'nit', age2 = 10)
```

```
nit
20
10
```

```
x = 27
y = 4
print(x % y * 3)
```

```
9
```

default argument

- while you open meta account minumu age critierial is so. by default age is 18

```
def person(name,age): #in this code we expected to print 2 but we got bydefault
    print(name)
    print(age)

person('nit')
```

```
-----
-----
TypeError                                Traceback (most recent call
last)
```

```
Cell In[50], line 5
      2     print(name)
      3     print(age)
----> 5 person('nit')
```

```
TypeError: person() missing 1 required positional argument: 'age'
```

```
def person(name,age=18):
    print(name)
    print(age)
```

```
person('nit',20)
```

```
nit
20
```

```
def person(name,age = 18):
    print(name)
    print(age)
```

```
person('nit',24) #in hear bydefault override the existing default value
```

```
nit
24
```

variable length argument image.png

```
def sum(a, b):
    c = a+b
    print(c)
```

```
sum(5,6)
```

```
11
```

```
def sum(a, b):  
    c = a+b  
    print(c)
```

```
sum(5,6,7,8)
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
Cell In[54], line 5  
      2     c = a+b  
      3     print(c)  
----> 5 sum(5,6,7,8)
```

TypeError: sum() takes 2 positional arguments but 4 were given

```
def sum(a, *b): # 1st argument is fixed but for 2nd argument  
    c = a+b  
    print(c)
```

```
sum(5,6,7,8)
```

we got error as int & tuple error becuae a is integer & b is tuple

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)
```

```
Cell In[55], line 5  
      2     c = a+b  
      3     print(c)  
----> 5 sum(5,6,7,8)  
      6 # we got error as int & tuple error becuae a is integer & b  
is tuple
```

```
Cell In[55], line 2, in sum(a, *b)  
      1 def sum(a, *b): # 1st argument is fixed but for 2nd argument  
----> 2     c = a+b  
      3     print(c)
```

TypeError: unsupported operand type(s) for +: 'int' and 'tuple'

```
def sum(a, *b): # 1st argument is fixed but for 2nd argument  
    #c = a+b  
    print(type(a))  
    print(type(b))
```

```
sum(5,6,7,8)
```

```
<class 'int'>  
<class 'tuple'>
```

```
def sum(a, *b): # 1st argument is fixed but for 2nd argument
    #c = a+b
    print(a)
    print(b)
```

```
sum(5,6,7,8)
```

```
5
(6, 7, 8)
```

```
def sum(a, *b): # 1st argument is fixed & we fetch each value from the
tuple & we can add them.
```

```
    c = a
    for i in b:
        c = c + i
    print(c)
```

```
sum(5,6,7,8)
```

```
26
```

```
def sum(a, *b):
```

```
    c = a
    for i in b:
        c = c + i
    print(c)
```

```
sum(5,6,7,8,9)
```

```
35
```

```
def sum(a, *b):
```

```
    c = 0
    for i in b:
        c = c + i
    print(c)
```

```
sum(5,6,7,8)
```

```
21
```

```
def sum(a, *b):
```

```
    c = 0
    for i in b:
        c = c + i
    print(c)
```



```
sum(5,6,7,8,5,7,7,89,-100,23)
```

52

- KWARGS (key worded variable length arguments)

```
def person():  
    person('ALEX', 36, 'JOHN', 987767)
```

```
def person(name,*data):  
    print(name)  
    print(data)
```

```
person('ALEX', 36, 'JOHN', 987767)
```

```
ALEX  
(36, 'JOHN', 987767)
```

```
def person(name,*data):  
    print('name')  
    print(data)
```

```
person('ALEX', age = 36, home_place = 'southcity', mob = 987767)  
# we got error as keyword argument thats why we add another *
```

```
-----  
-----  
TypeError                                Traceback (most recent call  
last)  
Cell In[64], line 5  
      2     print('name')  
      3     print(data)  
----> 5 person('ALEX', age = 36, home_place = 'southcity', mob = 987767)  
      6 # we got error as keyword argument thats why we add another *
```

```
TypeError: person() got an unexpected keyword argument 'age'
```

```
def person(name, **data):  
    print(name)  
    print(data)
```

```
person('mark', age = 36, home_place = 'southcity', mob = 987767)
```

```
mark  
{'age': 36, 'home_place': 'southcity', 'mob': 987767}
```

```
def person(name,**data):  
    print('name')  
    print(data)
```

```
person('mark', age = 36, home_place = 'southcity', mob = 987767,
```

```

edu='phd', actor = 'john')
#even though you can keep add the keyword this concept we called as
KWARGS

name
{'age': 36, 'home_place': 'southcity', 'mob': 987767, 'edu': 'phd',
'actor': 'john'}

def person(name, **data):
    print(name)

    for i, j in data.items():
        print(i, j)

person('john', age = 36, home_place = 'southcity', mob = 987767, place =
'USA')

john
age 36
home_place southcity
mob 987767
place USA

```

Global variable vs Local Variable

- We discussed about variable & discussed about function
- Sometimes we are creating variable inside the function and sometimes we are creating variable outside of the function
- when we declare variable outside of the function that concept called as scope

```

a = 10

print(a)

10

a = 10 #-- global variable

def something():
    b = 15 #local variable
    print('in function',b)
    print('out function',a)

# in this code we are declaring 2 variable is this possible
# first line of a is called outside of the function
# inside the function is called local variable

a = 10

def something():
    b = 15

```

```

    print('in function',b)
print('out function',a)

# in this code we are declaring 2 variable is this possible
# first line of a is called outside of the function
# inside the function is called local variable
out function 10
a = 10

def something():
    a = 15

print('in function',a)
print('out function',a)

# in this code we are declaring 2 variable is this possible
# first line of a is called outside of the function
# inside the function is called local variable

in function 10
out function 10
a = 10

def something():
    a = 15 #hear a is local variable
    b = 8
    print(b)
    print(a)

#print(a)

#something()

a = 10

def something():
    a = 15 #hear a is local variable
    print(a)

print(a)

10

a = 10

def something():

```

```

    a = 15
    print('in function',a) # local variable

print('out function',a) # global variable

# In this code we didn't call the function

out function 10

a = 10

def something():
    b = 15
    print('in function',b) # local variable

something()

print('out function',a) # global variable

# 1st preference is always local variable

in function 15
out function 10

a = 10

def something():
    #if we remove this variable then can default it consider as global variable
    print('in function',a)

something()
print('out function',a)
# if we don't assign any variable inside the function by default both considered as local variable

in function 10
out function 10

a = 10

def something():
    b = 25
    #if we remove this variable then can default it consider as global variable
    print('in function',b)

something()
print('out function',a)
# if we don't assign any variable inside the function by default both considered as local variable

```

```
in function 25
out function 10
```

```
a = 10
```

```
def something():
    a = 55
    print('in function',a)
something()
```

```
print('out function',a)
```

```
in function 55
out function 10
```

if i want to define global variabel inside the function

```
a = 10
```

```
def something():
    global a
    b = 15 # 15 is converted to local when user assigned global a
    print('in function',b)
    print('gloabl variable', a)
```

```
something()
```

```
print('out function',a)
```

now in this case we dont have local variable & all variables are global variable only

so this is how we are assigned to local variabel & global variable

```
in function 15
gloabl variable 10
out function 10
```

```
a = 20
```

```
def something():
    global a
    a = 15 # we refered local to global
    print('in function',a)

    a = 9 # i want a to be local variable
    #can we assigned loca variabel in the function answer is not
cuz bydefault it will treate as global
    # can we declare local & gloabl inside th function
```

```
something()
```

```
print('out function',a)
```

```
in function 15
out function 9
```

```
import keyword
keyword.kwlist
```

```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',
 'continue',
 'def',
 'del',
 'elif',
 'else',
 'except',
 'finally',
 'for',
 'from',
 'global',
 'if',
 'import',
 'in',
 'is',
 'lambda',
 'nonlocal',
 'not',
 'or',
 'pass',
 'raise',
 'return',
 'try',
 'while',
 'with',
 'yield']
```

*# if we used local & global in the same function this is not good idea
thats wy introduced to GLOBALS*

```
a = 10
print(id(a))
```

```
def something():
    a = 9
```

```

x = globals()['a'] #gloabls can give you all the gloabls

print(id(x))
print('in function',a)

something()
print('out function',a)

140707061867720
140707061867720
in function 9
out function 10

# now lets introduce special function called globals & using globals
we can access global variable address

a = 10
print(id(a))

def something():
    a = 9
    x = globals() # if i dont mention a then it will creat new memory

    print(id(x))
    print('in function',a)

    globals()['a'] = 15

something()
print('out function',a)

140707061867720
2304086906432
in function 9
out function 15

```

global variable --

- The global keyword is used inside a function to indicate that a variable refers to a global variable, not a local one.

```

x = 10 # Global variable

def update_x():
    global x # Declare that we are using the global variable x
    x += 5 # Modify the global variable

```

```
update_x()
print(x) # Output: 15
15
```

globals -->

- globals() is a built-in function that returns a dictionary representing the current global symbol table. It allows you to access and modify global variables programmatically.

```
x = 10 # Global variable

def update_x():
    globals()['x'] += 6 # Access and modify the global variable using
the dictionary

update_x()
print(x) # Output: 15
16

def count(lst):
    even = 0
    odd = 0

    for i in lst:
        if i%2 == 0:
            even += 1
        else:
            odd +=1
    return even,odd

lst = [10, 9, 8, 50, 34, 100]
even, odd = count(lst)

print(even)
print(odd)

5
1

def count(lst):
    even = 0
    odd = 0

    for i in lst:
        if i%2 == 0:
            even += 1
```



```

        else:
            odd +=1
        return even,odd

lst = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,11,12,13]
even,odd = count(lst)

print("Even Number: {} and odd Number : {}".format(even,odd))
#format is function belongs to string & bydefault you need to pass any parameter

Even Number: 6 and odd Number : 7

```

As fib is not inbuilt function so we need to define the function for it

```

def fib(n):
    print(0)
    print(1)

fib(0)

# in the above code we can get the fibonacci series but if the number is large then it takes more time

0
1

def fib(n):
    print(0)
    print(1)
    print(1)
    print(2)
    print(3)
    print(5)

fib(0)

0
1
1
2
3
5

# in programmin we need to continue these process thats why we need to use loop hear

def fib(n):
    a = 0
    b = 1

```

```

print(a)
print(b)

for i in range(0, n):
    c = a + b
    a = b
    b = c

    print(c)

```

```
fib(10)
```

```

0
1
1
2
3
5
8
13
21
34
55
89

```

Ignore below code

'''if user wants 5 value then above code is applicable but if user wants only 1 value then if you write #fib(1) then you will get 2 vales thats why we need to write the condition hear.'''

```

def fib(n):
    a, b = 0, 1
    if n == 1:
        print(a)
    else:
        print(a)
        print(b)

        for i in range(2, n):
            c = a + b
            a = b
            b = c
            print(c)

```

```
fib(2)
```

```

0
1

```

Factorial number-->

```
def fact(n):  
    f = 1  
    for i in range(1, n+1):  
        f = f*i  
  
    return f
```

```
x = 4
result = fact(x)
print(result)
```

```
# please use debug the code in pycharm for more indetail explanation &
breakthrouth point at f = 1
```

24

```
#help()
```

RECURSSION - CALLING FUNCTION FROM ITSELF IS CALLED RECURSSION

```
def wish():  
    print('hello')  
    print('hi')
```

wish()

```
hello
hi
```

```
def wish():  
    print('hello')  
wish()
```

wish()

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

```
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
```

```
-----
-----
RecursionError                                Traceback (most recent call
last)
Cell In[95], line 4
      2     print('hello')
      3     wish()
----> 4 wish()

Cell In[95], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[95], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

[... skipping similar frames: wish at line 3 (2970 times)]

Cell In[95], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[95], line 2, in wish()
      1 def wish():
----> 2     print('hello')
      3     wish()

File ~\anaconda3\Lib\site-packages\IPython\core\
interactiveshell.py:3056, in
InteractiveShell._tee.<locals>.write(data, *args, **kwargs)
    3054 if not data:
```



```
3055     return result
-> 3056 execution_count = self.execution_count
3057 output_stream = None
3058 outputs_by_counter = self.history_manager.outputs
```

```
File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:687, in
TraitType.__get__(self, obj, cls)
    685     return self
    686 else:
--> 687     return t.cast(G, self.get(obj, cls))
```

```
File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:666, in
TraitType.get(self, obj, cls)
    664     raise TraitError("Unexpected error in TraitType: default
value not set properly") from e
    665 else:
--> 666     return t.cast(G, value)
```

RecursionError: maximum recursion depth exceeded

```
# i want to cal the hello multiple time
# it will execute maximum 1000 time & in below code wish is calling by
itself
# bydefault we have 1000 limitation can we extend the recurssion
limitation yes we can
```

```
def wish(): #-----> 2-greeting function will executed
    print('hello')
```

```
wish() # What if i call the function again #3-----> function calls
itself is called recurssion
```

```
wish() #-----> 1-at this point we are calling wish() function
```

```
# it will print infinity time cuz recursion its own function
```

```
hello
hello
```

```
def wish():
    print('hello')
    wish()
```

```
wish()
```

```
hello
hello
hello
hello
hello
hello
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible][illegible]

Cell In[97], line 5

```
2 print('hello')
3 wish()
```

```
----> 5 wish()
```

```
Cell In[97], line 3, in wish()
```

```
1 def wish():
2     print('hello')
```

```
----> 3      wish()
```

```
Cell In[97], line 3, in wish()
```

```
1 def wish():
2     print('hello')
```

```
----> 3      wish()
```

```
[... skipping similar frames: wish at line 3 (2970 times)]
```

```
Cell In[97], line 3, in wish()
```

```
1 def wish():
2     print('hello')
```

```
----> 3 wish()
```

```
Cell In[97], line 2, in wish()
      1 def wish():
----> 2     print('hello')
      3     wish()
```

```
File ~\anaconda3\Lib\site-packages\IPython\core\
interactiveshell.py:3056, in
InteractiveShell._tee.<locals>.write(data, *args, **kwargs)
    3054 if not data:
    3055     return result
-> 3056 execution_count = self.execution_count
    3057 output_stream = None
    3058 outputs_by_counter = self.history_manager.outputs
```

```
File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:687, in
TraitType.__get__(self, obj, cls)
    685     return self
    686 else:
--> 687     return t.cast(G, self.get(obj, cls))
```

```
File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:666, in
TraitType.get(self, obj, cls)
    664     raise TraitError("Unexpected error in TraitType: default
value not set properly") from e
    665 else:
--> 666     return t.cast(G, value)
```

RecursionError: maximum recursion depth exceeded

```
import sys
sys.getrecursionlimit()
```

3000

```
import sys
sys.setrecursionlimit(200)
print(sys.getrecursionlimit())
```

200

```
import sys
sys.getrecursionlimit()
```

200

```
def wish():
    print('hello')
    wish()
wish()
```

[illegible]

[illegible]

[illegible]

[illegible][illegible]

```
Cell In[101], line 4
      2     print('hello')
      3     wish()
----> 4 wish()
```

```
Cell In[101], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()
```

```
Cell In[101], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()
```

```
[... skipping similar frames: wish at line 3 (170 times)]
```

```
Cell In[101], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()
```

Cell In[101], line 2, in wish()

```
1 def wish():  
----> 2     print('hello')  
      3     wish()
```

File ~\anaconda3\Lib\site-packages\IPython\core\interactiveshell.py:3056, in InteractiveShell._tee.<locals>.write(data, *args, **kwargs)
3054 if not data:
3055 return result
-> 3056 execution_count = self.execution_count
3057 output_stream = None
3058 outputs_by_counter = self.history_manager.outputs

File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:687, in TraitType.__get__(self, obj, cls)
685 return self
686 else:
--> 687 return t.cast(G, self.get(obj, cls))

File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:666, in TraitType.get(self, obj, cls)
664 raise TraitError("Unexpected error in TraitType: default value not set properly") from e
665 else:
--> 666 return t.cast(G, value)

RecursionError: maximum recursion depth exceeded

```
import sys  
sys.getrecursionlimit()  
  
200  
  
sys.setrecursionlimit(100)  
sys.getrecursionlimit()  
  
100  
  
import sys  
sys.setrecursionlimit(100)  
print(sys.getrecursionlimit())  
  
i = 0  
  
def wish():  
    global i  
    i += 1  
    print('hello', i)
```

```
wish()  
wish()  
# how to know how many wish it printed  
# so for best practice i would suggest for
```

```
100  
hello 1  
hello 2  
hello 3  
hello 4  
hello 5  
hello 6  
hello 7  
hello 8  
hello 9  
hello 10  
hello 11  
hello 12  
hello 13  
hello 14  
hello 15  
hello 16  
hello 17  
hello 18  
hello 19  
hello 20  
hello 21  
hello 22  
hello 23  
hello 24  
hello 25  
hello 26  
hello 27  
hello 28  
hello 29  
hello 30  
hello 31  
hello 32  
hello 33  
hello 34  
hello 35  
hello 36  
hello 37  
hello 38  
hello 39  
hello 40  
hello 41  
hello 42  
hello 43  
hello 44
```

```
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
hello 56
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello
```

```
-----
-----
RecursionError                                Traceback (most recent call
last)
```

```
Cell In[105], line 12
     10     print('hello', i)
     11     wish()
--> 12 wish()
     13 # how to know how many wish it printed
     14 # so for best practice i would suggest for
```

```
Cell In[105], line 11, in wish()
     9 i += 1
    10 print('hello', i)
--> 11 wish()
```

```
Cell In[105], line 11, in wish()
     9 i += 1
    10 print('hello', i)
```

```
---> 11 wish()
```

```
[... skipping similar frames: wish at line 11 (70 times)]
```

```
Cell In[105], line 11, in wish()
```

```
    9 i += 1
    10 print('hello', i)
---> 11 wish()
```

```
Cell In[105], line 10, in wish()
```

```
    8 global i
    9 i += 1
---> 10 print('hello', i)
    11 wish()
```

```
File ~\anaconda3\Lib\site-packages\IPython\core\
interactiveshell.py:3056, in
InteractiveShell._tee.<locals>.write(data, *args, **kwargs)
```

```
    3054 if not data:
    3055     return result
-> 3056 execution_count = self.execution_count
    3057 output_stream = None
    3058 outputs_by_counter = self.history_manager.outputs
```

```
File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:687, in
TraitType.__get__(self, obj, cls)
```

```
    685     return self
    686 else:
--> 687     return t.cast(G, self.get(obj, cls))
```

```
File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:666, in
TraitType.get(self, obj, cls)
```

```
    664     raise TraitError("Unexpected error in TraitType: default
value not set properly") from e
    665 else:
--> 666     return t.cast(G, value)
```

```
RecursionError: maximum recursion depth exceeded
```

```
i = 0
```

```
def wish():
    global i
    i += 1
    print('hello', i)
    wish()
wish()
```

```
# how to know how many wish it printed
# so for best practice i would suggest for
```

```
hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
hello 39
hello 40
hello 41
hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
hello 50
```



```
hello 51
hello 52
hello 53
hello 54
hello 55
hello 56
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello
```

```
-----
-----
RecursionError                                Traceback (most recent call
last)
```

```
Cell In[106], line 8
      6     print('hello', i)
      7     wish()
----> 8 wish()
     10 # how to know how many wish it printed
     11 # so for best practice i would suggest for
```

```
Cell In[106], line 7, in wish()
      5 i += 1
      6 print('hello', i)
----> 7 wish()
```

```
Cell In[106], line 7, in wish()
      5 i += 1
      6 print('hello', i)
----> 7 wish()
```

```
[... skipping similar frames: wish at line 7 (70 times)]
```

```
Cell In[106], line 7, in wish()
      5 i += 1
```

```
    6 print('hello', i)
----> 7 wish()
```

Cell In[106], line 6, in wish()

```
    4 global i
    5 i += 1
----> 6 print('hello', i)
    7 wish()
```

File ~\anaconda3\Lib\site-packages\IPython\core\interactiveshell.py:3056, in InteractiveShell._tee.<locals>.write(data, *args, **kwargs)

```
    3054 if not data:
    3055     return result
-> 3056 execution_count = self.execution_count
    3057 output_stream = None
    3058 outputs_by_counter = self.history_manager.outputs
```

File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:687, in TraitType.__get__(self, obj, cls)

```
    685     return self
    686 else:
--> 687     return t.cast(G, self.get(obj, cls))
```

File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:666, in TraitType.get(self, obj, cls)

```
    664     raise TraitError("Unexpected error in TraitType: default
value not set properly") from e
    665 else:
--> 666     return t.cast(G, value)
```

RecursionError: maximum recursion depth exceeded

```
import sys
sys.getrecursionlimit()
```

100

```
def wish():
    print('hello')
    wish()
wish()
```

```
hello
hello
hello
hello
hello
hello
hello
hello
hello
```

[illegible]

```
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
hello
```

```
-----
-----
RecursionError                                Traceback (most recent call
last)
Cell In[108], line 4
      2     print('hello')
      3     wish()
----> 4 wish()

Cell In[108], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[108], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

[... skipping similar frames: wish at line 3 (70 times)]

Cell In[108], line 3, in wish()
      1 def wish():
      2     print('hello')
----> 3     wish()

Cell In[108], line 2, in wish()
      1 def wish():
----> 2     print('hello')
      3     wish()

File ~\anaconda3\Lib\site-packages\IPython\core\
```

```

interactiveshell.py:3056, in
InteractiveShell._tee.<locals>.write(data, *args, **kwargs)
    3054 if not data:
    3055     return result
-> 3056 execution_count = self.execution_count
    3057 output_stream = None
    3058 outputs_by_counter = self.history_manager.outputs

File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:687, in
TraitType.__get__(self, obj, cls)
    685     return self
    686 else:
--> 687     return t.cast(G, self.get(obj, cls))

File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:666, in
TraitType.get(self, obj, cls)
    664     raise TraitError("Unexpected error in TraitType: default
value not set properly") from e
    665 else:
--> 666     return t.cast(G, value)

```

RecursionError: maximum recursion depth exceeded

```

import sys
print(sys.getrecursionlimit())

100

sys.setrecursionlimit(1000)
print(sys.getrecursionlimit())

1000

import sys
sys.setrecursionlimit(150)
print(sys.getrecursionlimit())

i = 0

def wish():
    global i
    i += 1
    print('hello', i)
    wish()
wish()

# how to know how many wish it printed
# so for best practice i would suggest for

```

```
150
hello 1
hello 2
hello 3
hello 4
hello 5
hello 6
hello 7
hello 8
hello 9
hello 10
hello 11
hello 12
hello 13
hello 14
hello 15
hello 16
hello 17
hello 18
hello 19
hello 20
hello 21
hello 22
hello 23
hello 24
hello 25
hello 26
hello 27
hello 28
hello 29
hello 30
hello 31
hello 32
hello 33
hello 34
hello 35
hello 36
hello 37
hello 38
hello 39
hello 40
hello 41
hello 42
hello 43
hello 44
hello 45
hello 46
hello 47
hello 48
hello 49
```

hello 50
hello 51
hello 52
hello 53
hello 54
hello 55
hello 56
hello 57
hello 58
hello 59
hello 60
hello 61
hello 62
hello 63
hello 64
hello 65
hello 66
hello 67
hello 68
hello 69
hello 70
hello 71
hello 72
hello 73
hello 74
hello 75
hello 76
hello 77
hello 78
hello 79
hello 80
hello 81
hello 82
hello 83
hello 84
hello 85
hello 86
hello 87
hello 88
hello 89
hello 90
hello 91
hello 92
hello 93
hello 94
hello 95
hello 96
hello 97
hello 98
hello 99

```
hello 100
hello 101
hello 102
hello 103
hello 104
hello 105
hello 106
hello 107
hello 108
hello 109
hello 110
hello 111
hello 112
hello 113
hello 114
hello 115
hello 116
hello 117
hello 118
hello 119
hello 120
hello 121
hello 122
hello 123
hello
```

```
-----
-----
RecursionError                                Traceback (most recent call
last)
Cell In[112], line 12
     10     print('hello', i)
     11     wish()
--> 12 wish()
     14 # how to know how many wish it printed
     15 # so for best practice i would suggest for

Cell In[112], line 11, in wish()
      9 i += 1
     10 print('hello', i)
--> 11 wish()

Cell In[112], line 11, in wish()
      9 i += 1
     10 print('hello', i)
--> 11 wish()

[... skipping similar frames: wish at line 11 (120 times)]

Cell In[112], line 11, in wish()
```



```

    9 i += 1
    10 print('hello', i)
--> 11 wish()

Cell In[112], line 10, in wish()
    8 global i
    9 i += 1
--> 10 print('hello', i)
    11 wish()

File ~\anaconda3\Lib\site-packages\IPython\core\
interactiveshell.py:3056, in
InteractiveShell._tee.<locals>.write(data, *args, **kwargs)
    3054 if not data:
    3055     return result
-> 3056 execution_count = self.execution_count
    3057 output_stream = None
    3058 outputs_by_counter = self.history_manager.outputs

File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:687, in
TraitType.__get__(self, obj, cls)
    685     return self
    686 else:
--> 687     return t.cast(G, self.get(obj, cls))

File ~\anaconda3\Lib\site-packages\traitlets\traitlets.py:666, in
TraitType.get(self, obj, cls)
    664     raise TraitError("Unexpected error in TraitType: default
value not set properly") from e
    665 else:
--> 666     return t.cast(G, value)

RecursionError: maximum recursion depth exceeded

```

FACTORIAL USING RECURSSION

- recursion is function calls itself

```

def fact(n):
    if n==0:
        return 1
    return n * fact(n-1)

result = fact(5)

result

120

```

Function without name is called - ANONYMOUS FUNCTION OR LAMBDA

```
def square(a):  
    return a * a  
  
square(5)  
# what if i dont want to call square() multiple times  
  
25  
  
def square(a):  
    return a * a  
  
result = square(5)  
print(result)  
  
# what if i dont want to call square() multiple times  
  
25  
  
#lambda expresion or lambda function  
  
f = lambda a : a * a # hear a is an argument & operation in the  
argument is a * a  
result = f(5)  
result  
  
# hear anonymous function is called lambda  
# remember lambda alway you need to assgin as function cuz function  
are object in python  
  
25  
  
f = lambda a, b : a + b  
f1 = lambda a, b: a-b  
  
result = f(1,4)  
result1 = f1(4,1)  
  
print(result)  
print(result1)  
  
5  
3  
  
#lets define function which will add 2 number  
# we are defining a function whcih doesnt have name  
#lambda expresion or lambda function  
  
f = lambda a, b : a + b  
f1 = lambda a, b : a - b
```

```
result = f(1,4)
result1 = f1(2, 3)
```

```
print(result)
print(result1)
```

```
5
-1
```

```
import keyword
keyword.kwlist
```

```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',
 'continue',
 'def',
 'del',
 'elif',
 'else',
 'except',
 'finally',
 'for',
 'from',
 'global',
 'if',
 'import',
 'in',
 'is',
 'lambda',
 'nonlocal',
 'not',
 'or',
 'pass',
 'raise',
 'return',
 'try',
 'while',
 'with',
 'yield']
```

How can we use lambda in other function like - filter, map & reduce

#lets take one list & i want to find the list of even numbers

```
nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(is_even, nums)) #is_even is not an inbuild function
```


NameError Traceback (most recent call last)

Cell In[120], line 4

```
1 #lets take one list & i want to find the list of even numbers
```

```
2 nums = [3,2,6,8,4,6,2,9]
```

```
----> 4 evens = list(filter(is_even, nums)) #is_even is not an inbuild function
```

NameError: name 'is_even' is not defined

```
def is_even(n):  
    return n % 2 == 0
```

```
nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(is_even, nums))
```

```
print(evens)
```

```
[2, 6, 8, 4, 6, 2]
```

```
def is_odd(n):  
    return n % 2 != 0
```

```
nums = [3,2,6,8,4,6,2,9]
```

```
odd = list(filter(is_odd, nums))
```

```
print(odd)
```

```
[3, 9]
```

lets write above function using help of lambda & lambda helps to reduce the line for better optimization

```
nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(lambda n : n%2 == 0, nums))
```

```
print(evens)
```

```
[2, 6, 8, 4, 6, 2]
```

```
nums = [3,2,6,8,4,6,2,9]
```

```
odd = list(filter(lambda n : n%2 !=0, nums))
```

```
print(odd)
```

```
[3, 9]
```

lets write above function using help of lambda & lambda helps to reduce the line

```
nums = [3,2,6,8,4,6,2,9, 10, 13]
```

```
evens = list(filter(lambda n : n%2 ==0, nums))
```

```
odd = list(filter(lambda n : n%2 !=0, nums))
```

```
print(evens)
```

```
print(odd)
```

```
[2, 6, 8, 4, 6, 2, 10]
```

```
[3, 9, 13]
```

- What ever even number I have from the assigned list
- I want to double the even number i.e 2 become 4 || 4 become 6 || 6 become 8
- that we will do using map function
- this largely we are using in google map reduce programme
- we can build using user define & lambda

```
def update(n):  
    return n-2
```

```
nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(is_even, nums))
```

```
double = list(map(update, evens))
```

```
print(evens)
```

```
print(double)
```

```
[2, 6, 8, 4, 6, 2]
```

```
[0, 4, 6, 2, 4, 0]
```

```
nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(is_even, nums))
```

```
double = list(map(lambda n : n*2, evens))
```

```
print(evens)
```

```
print(double)
```

```
[2, 6, 8, 4, 6, 2]
```

```
[4, 12, 16, 8, 12, 4]
```

```
nums = [3,2,6,8,4,6,2,9]
```

```
evens = list(filter(is_even, nums))
```

```
double = list(map(lambda n : n*2, evens))
```

```
double_ = list(map(lambda n : n+2, evens))
```

```
doubble_1 = list(map(lambda n : n-2, evens))
```

```

print(evens)
print(double)
print(double_)
print(doubble_1)

[2, 6, 8, 4, 6, 2]
[4, 12, 16, 8, 12, 4]
[4, 8, 10, 6, 8, 4]
[0, 4, 6, 2, 4, 0]

nums = [3,2,6,8,4,6,2,9]

evens = list(filter(is_even, nums))

double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n-2, evens))

print(double)
print(double_)

[4, 12, 16, 8, 12, 4]
[0, 4, 6, 2, 4, 0]

nums = [3,2,6,8,4,6,2,9]
evens = list(filter(is_even, nums))

double = list(map(lambda n : n*2, evens))
double_ = list(map(lambda n : n-2, evens))
double1 = list(map(lambda n : n+2, evens))

print(double)
print(double_)
print(double1)

[4, 12, 16, 8, 12, 4]
[0, 4, 6, 2, 4, 0]
[4, 8, 10, 6, 8, 4]

```

- i want to perform reduce now
- i want reduce all the values
- reduce you can add only 2 values
- [4, 12, 16, 8, 12, 4] if you sum everything then you will get 56

```

from functools import reduce

def add_all(a,b):
    return a+b

nums = [3,2,6,8,4,6,2]

```

```
evens = list(filter(is_even, nums))
double = list(map(lambda n : n*2, evens))

sums = reduce(add_all, double)
sums
print(sums)

56

a = [7,8]
print(type(a))

<class 'list'>
```

python decorators

- This is one of amazin feature in python
- We knew that function are build on certain task
- lets understand we have some predefined function
- div() function which takes 2 parameter

```
def div(a,b):
    print(a // b)
div(4,2)
# but what if we pass the value 2, 4

2

def div(a, b):
    print(a // b)
div(2,4)

0
```

- i want different logic hear
- it does not matter in which sequence i pass the value alway numerator should be greather then denominator (numerator > denominator)
- in the above function when i divide 2/4 (numerator < denominator)
- even though user pass 2/4 but inbuild it should consider as 4 / 2 but how to swap lets discuss
- before division 2/4 lets apply the logic & logic is if 2<4 then i want to swap

```
def div(a,b):

    if a<b:
        a,b = b,a
    print(a // b)

div(2, 4)
```

- lets understand the twist, imagine the code you dont have , you work in the organization
- inbuild the code & you dont want to change the existing code
- without impacting on existing code you need to get the solution
- being a developer you need to swap the value without touching the existing function
- thats the reason DECORATOR comes into the picture
- using the help of DECORATOR you can add the extra feature in the existing function
- lets create a new function for decorator
- we can write functin inside the funtion thats why we called as python is functional programming language

```
# using help of the decorator you can add the extra feature in the  
existing function
```

```
def div(a,b):
    print(a / b)

def div_decorator(func): # hear div_decorator will accept the div function
    def inner(a,b):
        if a<b:
            a,b = b,a
        return func(a,b)
    return inner

div = div_decorator(div)

div(2,4)

2.0

def my_decorator(func):
    def wrapper():
        print("Something is happening before the function is called.")
        #func()
        print("Something is happening after the function is called.")
    return wrapper

#@my_decorator
#def say_hello():
#    print("Hello!")

say_hello()
```

[illegible]


```

Cell In[137], line 12
      6     return wrapper
      8     #@my_decorator
      9     #def say_hello():
     10     #     print("Hello!")
--> 12 say_hello()

```

NameError: name 'say_hello' is not defined

```

def my_decorator(func):
    def wrapper():
        print("Something is happening before the function is called.")
        func()
        print("Something is happening after the function is called.")
    return wrapper

```

```

@my_decorator
def say_hello():
    print("Hello!")

```

```
say_hello()
```

Something is happening before the function is called.

Hello!

Something is happening after the function is called.

- the best part of the decorator is you can change the behaviour of existing function at the time of compiler
- what we done now we take new function which take func as an parameter
- you cant do like this any programming language
- we can defin the function with in the function which actually replacing using swap
- without changing existing code with the help of decorator we are get the output
- concept of debugging means - remov the bugs
- what is codeing - coding will create bugs or add the bugs
- Brekdwn complex codes or logical part to different modules
- if you see the image
- lets break down to logical part ac is 1 module & bd is another module
- again we are breaking into c module, a module, b module , d module as well
- main advantage is if you change module a it wont effect on other module b, or c
- another advantage is you can re-use the module
- if you build other module as GADP (A & D) module is reused
- Inbuild module (import numpy as np)
- user define module as well (lets create how to define user define module)

- for that lets take one new file as test.ipynb which define 4 function (add, sub, multi, div)
- i want to use this function in another module
- so lets create our own module

```
__name__
'__main__'
```

Special Variable **name**

```
__name__
'__main__'
print(__name__)
__main__
```

- The moment when we print **name** then i will get output as main
- in your project if you have 5 modules or 10 modules there will some modules which will be run first
- so the frist module name is always **main**. thats why the code start
- to understand indetaild review we will work with PYCHARM
- from the main function we can call all other function
- please work 4 file in pycharm (hi.py, bye.py, hi1-specialvariable, bye1-specialvariable)
- refer more about chatgpt